Authentication and Security

This document covers the authentication mechanisms, security practices, and data protection measures applied throughout the system.

Kiosk Authentication

Admin Password

The kiosk's admin panel is protected by a password, stored as a hash in the `admins` table. Older installs stored this as an unsalted SHA-256 digest; the app now hashes new passwords with `werkzeug.security.generate_password_hash` (salted PBKDF2, the same scheme the dashboard uses for `dashboard_users` -- see below) instead.

`check_admin_password()` transparently upgrades an existing legacy hash the first time it's used successfully, so an already-deployed kiosk keeps working with zero manual steps after an update -- this is a "drop and play" requirement, since the kiosk is a physical machine with no remote password reset:

```
def check_admin_password(password: str) -> bool:
    with get_conn() as conn:
        row = conn.execute("SELECT id, password_hash FROM admins WHERE username='admin'").fetchone()
    if not row:
        return False
    stored = row["password_hash"]
    if ":" in stored:                          # already a werkzeug hash
        return check_password_hash(stored, password)
    if hashlib.sha256(password.encode()).hexdigest() != stored:  # legacy format
        return False
    # correct legacy password -- silently re-hash in the new format
    with get_conn() as conn:
        conn.execute("UPDATE admins SET password_hash=? WHERE id=?",
                     (generate_password_hash(password), row["id"]))
    return True
```

Werkzeug's hash strings always contain a `:` delimiter (e.g. `pbkdf2:sha256:...`), while a raw SHA-256 hex digest never does -- that's the cheap, reliable way to tell the two formats apart without a separate "hash version" column. `set_admin_password()` (used when someone deliberately changes the password) always writes the new format directly.

Hashing

Hashing is a one-way function: it is computationally infeasible to recover the original password from its hash. This means that even if the database file is accessed directly, the admin password cannot be read.

Salted PBKDF2 hashing (via `werkzeug.security`) additionally protects against precomputed rainbow-table attacks and is deliberately slow to compute, which raw SHA-256 is not -- SHA-256 was designed for speed, which is actually a weakness for password hashing since it makes brute-forcing cheap.

Default Credentials

The system seeds a default admin account on first run: - Username: `admin` - Password: `admin1234`

The password should be changed through the admin panel after initial setup.

Member PIN Verification

Members can optionally have a PIN set on their profile. The `verify_pin()` function checks the entered PIN against the stored one:

```
def verify_pin(member_id: int, pin: str) -> bool:
    stored = row["pin"] or ""
    return stored == "" or stored == pin
```

A blank PIN means no restriction -- anyone can sign the member in without entering a PIN. This is a deliberate design choice for simplicity in a childcare environment where caregivers may not always have a PIN available.

Current status: the PIN can still be set and stored on a member's or staff member's profile, and `verify_pin()` is kept (and unit tested) in `database.py`, but neither the kiosk sign-in flow nor the web Sign In / Out page currently asks for a PIN or calls `verify_pin()`. Selecting a person is enough to sign them in or out.

Dashboard Authentication

The dashboard is reachable over the internet (e.g. tunnelled via Cloudflare Tunnel or a similar reverse tunnel that terminates HTTPS/TLS at its edge), so its authentication is designed to stand on its own rather than relying on being LAN-only.

Multi-User Login: Name + Password, With Roles

Dashboard logins are no longer a single hardcoded credential. Every account lives in a `dashboard_users` table (see 02-database-design.md) with a `name`, a hashed `pin_hash`, a `role` of either `admin` or `staff`, an optional `member_id` link, and lockout state:

```
CREATE TABLE dashboard_users (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    name            TEXT UNIQUE NOT NULL,
    pin_hash        TEXT NOT NULL,
    role            TEXT NOT NULL CHECK(role IN ('admin', 'staff')),
    created_at      TEXT DEFAULT (datetime('now', 'localtime')),
    member_id       INTEGER REFERENCES members(id),
    failed_attempts INTEGER DEFAULT 0,
    locked_until    TEXT
);
```

The `pin_hash` column name is unchanged from when this table only stored numeric PINs -- it now stores a full password hash, but renaming the column would add migration risk for no functional benefit, so only the

validation rules and the "PIN" wording changed, not the schema's column name. Passwords are hashed with `werkzeug.security.generate_password_hash` (PBKDF2-based, salted) -- `check_dashboard_login()` verifies with `check_password_hash()`, which is timing-safe. On first run, a single account is seeded (`name="admin"`, password `1234`, `role="admin"`) so the live dashboard always has a working login; a console warning is printed until that password is changed via the "Manage Dashboard Users" admin page. The seeded password does not meet the rules below -- it predates them, and existing/seeded credentials are never retroactively invalidated (see Password Requirements).

An **admin** account is created by an existing admin from the Manage Dashboard Users page; **staff** accounts log in with just their name and password and can reach the web Sign In/Out page, the read-only `/status` ("Who's Here") page, and (if their account is linked to a staff record) their own "My Timesheet" page -- everything else (Dashboard analytics, People, Exports, Audit Log, Alerts, Manage Dashboard Users) requires `role == "admin"`.

Linking a Login to a Staff Record

`member_id` optionally links a `dashboard_users` row to a `members` row (`person_type='staff'`), set by an admin from the "Manage Dashboard Users" create/edit pages via a dropdown. This is what lets the dashboard resolve "which person's timesheet is this" for the self-service "My Timesheet" page (see 07-export-system.md) -- the person is always resolved from `session["member_id"]` server-side, never from a client-supplied id, so one staff account can never view or download another's sign-in history by tampering a URL.

Password Requirements

New and changed dashboard passwords must pass `validate_dashboard_password()`: - At least 10 characters (above the commonly cited 8-character floor, chosen because this app has a small, somewhat guessable set of account names and no throttling independent of IP beyond the rate limiter below -- see Account Lockout). - At least one digit. - At least one non-alphanumeric ("special") character.

No uppercase/lowercase mixing is required -- that rule tends to produce worse, more predictable chosen passwords (`Password1!`) for little real benefit. Validation runs only when a password is created or changed (account creation, or an admin setting a new password from the edit page) -- it is never checked at login, so an account created or last changed before this rule existed keeps logging in exactly as before. This is a deliberate "drop and play" requirement: dropping in an updated build must never lock out an existing login.

Session Contents and Route Protection

After a successful login, the session stores the authenticated user's identity, not just a boolean:

```
session["user_id"] = user["id"]
session["name"] = user["name"]
session["role"] = user["role"]
session["member_id"] = user.get("member_id")
session.permanent = True
```

`session["member_id"]` is what the dashboard nav bar checks to decide whether to show the "My Timesheet" link.

Two decorators protect routes:

```
def login_required(f):        # any authenticated user, any role
   ...
   if not session.get("user_id"):
       return redirect(url_for("login"))

def admin_required(f):        # authenticated AND role == "admin"
   ...
   if session.get("role") != "admin":
       flash("Admin access required.", "error")
       return redirect(url_for("signin"))
```

`login_required` alone guards the Sign In/Out page (`/signin`), the "Who's Here" status page (`/status`), and `/logout` -- any authenticated user, admin or staff, can reach these. `admin_required` guards every other page, including editing (`/logs/sign/<id>/edit`) or adding (`/logs/sign/add`) a sign-in/out time. A staff account that hits an admin-only URL directly is redirected back to `/signin` with a flash message, rather than a bare 403.

Sessions are signed cookies -- Flask's `secret_key` cryptographically signs them, so they cannot be forged or tampered with client-side without knowing that key.

Secret Key

The Flask `secret_key` is no longer a hardcoded string committed to source. It is read from the `DASHBOARD_SECRET_KEY` environment variable (optionally via a `.env` file, loaded with `python-dotenv` if installed):

```
def _resolve_secret_key():
   key = os.environ.get("DASHBOARD_SECRET_KEY")
   if not key:
       key = secrets.token_hex(32)
       print("WARNING: DASHBOARD_SECRET_KEY is not set ...")
   return key
```

If the variable is unset, a random key is generated for that process only -- the app still starts (rather than crashing or silently using a known key), but every session is invalidated on the next restart. Production deployments should set `DASHBOARD_SECRET_KEY` so sessions survive restarts. Note that changing the secret key (or the session schema, as happened in this migration) invalidates all existing sessions -- this is expected, not a bug; see 12-build-and-deployment.md.

CSRF Protection

All state-changing (`POST`) routes are protected by `flask-wtf`'s `CSRFProtect`, applied globally:

```
csrf = CSRFProtect(app)
```

Every HTML form that performs a `POST` (login, person create/edit/delete, alerts, dashboard user management, sign-in/out) includes a hidden `csrf_token` field. Requests missing or presenting an invalid token are rejected automatically -- there is no route-by-route opt-in to forget.

Login Rate Limiting

`flask-limiter` throttles the `/login` route to 5 attempts per minute per IP address:

```
@app.route("/login", methods=["GET", "POST"])
@limiter.limit("5 per minute")
def login():
    ...
```

This slows down credential-guessing without needing an external store -- the default in-memory limiter storage is sufficient since the dashboard runs as a single Waitress process (see below), not multiple worker processes that would each have their own counters.

Account Lockout

The IP rate limit above throttles a single source hammering `/login`, but doesn't stop a slow attacker or one spreading attempts across many IP addresses against one account name. `check_dashboard_login()` layers a second, account-specific defense on top: after `LOCKOUT_THRESHOLD` (5) consecutive failed attempts for a given name, that account is locked for `LOCKOUT_MINUTES` (15), tracked in `dashboard_users.failed_attempts`/`locked_until`:

```
LOCKOUT_THRESHOLD = 5
LOCKOUT_MINUTES = 15
```

While locked, `is_dashboard_user_locked()` makes `/login` refuse even the *correct* password with a distinct "Account temporarily locked" message, rather than the generic "Invalid name or password." A successful login resets `failed_attempts` to 0 and clears `locked_until`. The lockout always expires on its own (15 minutes after the last failed attempt to trip it) -- it is never permanent, since the kiosk's admin account has no remote way to be unlocked if it were.

Security Response Headers

Every dashboard response gets a standard set of hardening headers, set in a global `@app.after_request` hook -- no new dependency, no cost, and no effect on either the plain-HTTP LAN path or the HTTPS tunnel path:

```
response.headers["X-Content-Type-Options"] = "nosniff"
response.headers["X-Frame-Options"] = "DENY"
response.headers["Referrer-Policy"] = "same-origin"
response.headers["Content-Security-Policy"] = (
    "default-src 'self'; "
    "script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net; "
    "style-src 'self' 'unsafe-inline'; "
    "img-src 'self' data:"
)
```

- `X-Content-Type-Options: nosniff` stops the browser from guessing content types in a way that could turn an upload into executable script.

- `X-Frame-Options: DENY` prevents the dashboard from being embedded in an `<iframe>` on another site (clickjacking protection).
- `Referrer-Policy: same-origin` keeps the current URL out of the `Referer` header sent to any third-party resource.
- The `Content-Security-Policy` restricts where scripts/styles/images can load from. `'unsafe-inline'` is included deliberately: `dashboard.html` and `alerts.html` inject server-rendered chart/alert data via inline `<script>` blocks, and several templates use inline `style=""` attributes -- every inline script or style is this app's own template output, never user-supplied, so allowing it is a pragmatic choice over threading CSP nonces through the whole template set for no real security gain here. `https://cdn.jsdelivr.net` is whitelisted because that's where Chart.js loads from on the analytics dashboard.

`Strict-Transport-Security` (HSTS) is added only when `request.is_secure` is true, matching the same conditional pattern already used for `SESSION_COOKIE_SECURE` above -- forcing it unconditionally would tell browsers to always use HTTPS for this host, breaking the plain-HTTP LAN access path this app deliberately supports.

Secure Cookie Configuration

```
_cookie_secure = os.environ.get("DASHBOARD_COOKIE_SECURE", "false").strip().lower() == "true"

app.config.update(
    SESSION_COOKIE_HTTPONLY=True,
    SESSION_COOKIE_SAMESITE="Lax",
    SESSION_COOKIE_SECURE=_cookie_secure,
    PERMANENT_SESSION_LIFETIME=timedelta(hours=8),
)
```

`SESSION_COOKIE_SECURE=True` would make the browser refuse to store or send the session cookie at all over a plain HTTP connection -- safe behind a reverse tunnel, where the browser always sees an `https://` URL, but the dashboard is *also* documented as reachable directly over plain HTTP on the local network (staff/admins on the same Wi-Fi as the kiosk -- see the README and 12-build-and-deployment.md). Forcing `Secure` unconditionally broke every LAN login: with no session, the CSRF token that Flask-WTF stores in it never gets set either, so submitting the login form failed with "The CSRF session token is missing." rather than a plain "invalid password."

Since one running process serves both access paths at once, it can't statically be "Secure-only" without breaking one of them -- so this defaults to `False` (`DASHBOARD_COOKIE_SECURE` unset or anything other than `true`), which keeps LAN access working. The internet-facing path is still protected by the tunnel's own TLS termination regardless of this flag; `SESSION_COOKIE_SECURE` is a defense-in-depth extra on top of that, not the thing standing between an attacker and the login form. Set `DASHBOARD_COOKIE_SECURE=true` only for a deployment that's exclusively reachable over HTTPS (no LAN-HTTP access at all).

Production WSGI Server

The dashboard used to run on Flask's built-in development server (`app.run(...)`), which Flask itself warns is not suitable for production or internet-facing use. It's been replaced with Waitress, a pure-Python production WSGI server, in both places the dashboard is started -- standalone (`dashboard/app.py`'s `__main__` block) and embedded in the kiosk (`kiosk_signin/app.py`'s `_start_dashboard()`, which launches it on a background thread). Waitress needs no extra infrastructure (no nginx/Docker) and, as a bonus, defaults to a small thread pool rather than the dev server's single-threaded handling. See 11-threading-and-concurrency.md for how this fits the existing threading model.

SQL Injection Prevention

All database queries use parameterised placeholders (`?`) with bound parameters:

```
conn.execute("SELECT * FROM members WHERE id=?", (member_id,))
```

This is the primary defence against SQL injection. The database driver handles escaping and quoting of parameter values, making it impossible for user input to be interpreted as SQL code.

This practice is applied consistently across both `database.py` and `dashboard/app.py`. Even dynamically constructed WHERE clauses in the dashboard (for filtering logs) use parameterised queries:

```
where_parts.append("s.direction = ?")
params.append(direction)
```

Input Validation

Form Validation

Both the kiosk and dashboard validate that required fields (first name, last name) are not empty before saving. The kiosk shows a `PopupOverlay` error, while the dashboard uses `flash()` messages.

Date Validation

Export date ranges are validated using `datetime.strptime()` to ensure they match the expected `YYYY-MM-DD` format before being passed to database queries. The single-person export (`/export/person`) treats its `date_from`/`date_to` query params as optional and does not strictly validate their format -- they are only ever used in parameterised `DATE(s.timestamp) >= ?` / `<= ?` comparisons, so a malformed value just matches no rows rather than posing an injection risk.

Direction Constraint

The `sign_ins` table enforces valid directions at the database level:

```
direction TEXT NOT NULL CHECK(direction IN ('IN', 'OUT'))
```

This is a database-level constraint that rejects any value other than "IN" or "OUT".

Data Integrity

Foreign Key Constraints

Foreign keys are enabled with `PRAGMA foreign_keys=ON` on every connection. Combined with `ON DELETE CASCADE` on all foreign key definitions, this ensures:

- Deleting a member automatically removes their sign-in records, caregiver details, and run sheet assignments.
- No orphaned records can exist in child tables.

Unique Constraints

Several tables enforce uniqueness: - `admins.username` -- prevents duplicate kiosk admin accounts. - `dashboard_users.name` -- prevents duplicate dashboard logins; the "Manage Dashboard Users" page surfaces this as a friendly "already exists" error rather than a raw database error. - `run_groups.name` -- prevents duplicate run sheet names; both the kiosk's "Save Run Sheet" and the dashboard's Run Sheets create/edit pages surface a collision as a friendly "already exists" error rather than a raw database error, including when renaming an existing run sheet to a name another one already has.

Transaction Safety

SQLite connections used with Python's `with` statement provide automatic transaction management: - Successful execution commits the transaction. - Any exception rolls back the transaction.

This ensures that multi-statement operations (e.g. creating a member and logging the event) either both succeed or both fail.

Audit Trail

All significant actions are recorded in the `audit_log` table via the `log_event()` function:

- Member and staff creation, updates, and deletion
- Sign-in and sign-out events (from both the kiosk and the web dashboard)
- Admin corrections to a sign-in/out time (`SIGN_TIME_EDITED`, recording the old and new timestamp)
- Admin additions of a missing sign-in/out time (`SIGN_TIME_ADDED`, recording the timestamp it was added as)
- Dashboard login and logout, and failed dashboard login attempts (`DASHBOARD_LOGIN_FAILED`, recording the attempted name but never the password) -- this is what makes brute-force or guessing attempts visible on the Audit Log page, rather than only ever showing legitimate activity

- Dashboard user creation, updates, and deletion (including run sheet create/rename/delete, which share the same `RUN_CREATED`/`RUN_UPDATED`/`RUN_DELETED` event types regardless of whether the change came from the kiosk or the dashboard)
- Data exports (of any format -- xlsx, PDF, or CSV)

The audit log is append-only -- there is no mechanism to delete or modify audit entries through the application. Each entry includes an event type, description, and timestamp.

This provides accountability and traceability for all system operations.

Error Handling Strategy

Silent Failures for Non-Critical Operations

Email notifications and audit logging use broad exception handling that silently catches errors:

```
try:
    # send email / log event
except Exception:
    pass
```

This is intentional for operations that should not disrupt the primary workflow. A failed email should not prevent a sign-in from being recorded.

User-Facing Errors

Operations that affect data (saving profiles, changing passwords, exporting) show explicit error messages to the user through `PopupOverlay` (kiosk) or `flash()` (dashboard).